

# [76] ACORN: Performant and Predicate-Agnostic Search over Vector Embeddings and Structured Data

## 요약

Patel et al.의 PACMMOD 2024 논문으로, 벡터 임베딩과 구조화된 데이터를 모두 처리하는 술어-무관(predicate-agnostic) 하이브리드 검색 시스템을 제시한다. 기존 시스템들은 특정 술어(예: "category = electronics")에 최적화되는 경향이 있었으나, ACORN은 임의의 술어 조합에 일반적으로 적용 가능한 접근법을 제안한다.

핵심 아이디어는 벡터 검색(approximate nearest neighbor search)과 메타데이터 필터링을 독립적으로 실행한 후, 효율적인 방식으로 결과를 병합(merge)하는 것이다. 특별히, 기존의 "필터-후-검색(filter-then-search)" 방식의 한계를 극복하고, 다양한 필터 조건에 동적으로 적응하는 "동적 병합(dynamic merging)" 전략을 제안한다.

주요 기여:

1. **술어 무관성:** 어떤 형태의 술어든 지원 (범위, 등호, 복합)
2. **동적 최적화:** 필터 선택도에 따라 검색 전략 자동 조정
3. **성능:** 기존 방법 대비 2-5 배 빠른 처리

본 논문과의 관계: Exqutor가 해결하는 핵심 문제와 정확히 일치한다. 범위 필터와 벡터 검색의 결합에서 최적의 실행 전략을 선택하는 방식이 ACORN의 방법론을 직접 참고한다.

## 상세분석

### 76.1 하이브리드 검색의 문제 정의

#### 시나리오:

데이터베이스: 전자제품 카탈로그  
 - 벡터: 제품 설명 임베딩 (1536 차원)  
 - 메타데이터: 가격, 카테고리, 별점, 재고 등

쿼리: "스마트폰과 유사한 제품 중 가격 < 500 달러"

#### 기존 접근의 문제:

#### 방식 1: 필터-후-검색 (Filter-then-Search)

1 단계: 가격 < 500 필터링  
 → 100,000 개 제품 중 10,000 개 남음 (선택도 10%)  
 2 단계: 남은 10,000 개 제품에서만 벡터 검색  
 → 빠름!

BUT: 선택도가 높으면?

1 단계: 가격 < 2000 필터링  
 → 100,000 개 중 90,000 개 남음 (선택도 90%)  
 2 단계: 남은 90,000 개 제품에서 벡터 검색  
 → 매우 느림!

#### 방식 2: 검색-후-필터 (Search-then-Filter)

1 단계: 모든 제품에서 상위 1000 개 유사 제품 검색  
 2 단계: 상위 1000 개에서 가격 < 500 필터  
 → 일반적으로 빠름

BUT: 필터 선택도가 매우 낮으면?

1 단계: 모든 제품에서 상위 1000 개 검색  
 2 단계: 상위 1000 개 중 필터 조건 만족하는 제품이 5 개만?  
 → 결과 부족, 다시 더 많이 검색 필요 (반복)

## 76.2 동적 병합(Dynamic Merging) 전략

### 핵심 개념:

```

├─ 벡터 검색 스트림
│   ├─ 제품 A (유사도: 0.95)
│   ├─ 제품 B (유사도: 0.92)
│   ├─ 제품 C (유사도: 0.90)
│   └─ ...
└─ 메타데이터 필터 스트림
    ├─ 제품 A (가격: 400, 만족)
    ├─ 제품 D (가격: 300, 만족)
    ├─ 제품 E (가격: 450, 만족)
    └─ ...
  
```

동적 병합:

현재까지 찾은 결과: A (유사도 0.95, 가격 0)

다음 후보: B (유사도 0.92) vs D (가격만족)

최소 유사도 임계값: B의 유사도가 D 검색에 필요한 수준보다 높으면 B 우선

### 알고리즘:

```

def dynamic_merge_search(query_vector, filter_predicate):
    vector_iter = vector_search_knn(query_vector) # 유사도순 스트림
    metadata_iter = scan_with_filter(filter_predicate) # 필터 만족하는 항목들

    results = []

    while len(results) < k:
        next_vector = vector_iter.peek() # 다음 유사 항목
        next_metadata = metadata_iter.peek() # 다음 필터 만족 항목

        # 동적 결정: 어느 쪽을 더 탐색할지?
        if should_continue_vector_search(next_vector.similarity):
            results.append(next_vector)
            vector_iter.advance()
        else:
            # 벡터 검색 중단, 메타데이터 필터로 전환
            break

    return results[:k]
  
```

## 76.3 선택도 기반 전략 선택

### 선택도 추정의 중요성:

선택도( $\sigma$ ) = 필터를 만족하는 행의 비율

$\sigma < 10\%$  (매우 낮음):

└─ Filter-then-Search 추천

1. 빠른 필터 실행 (90% 제거)
2. 작은 집합에서만 벡터 검색

$\sigma = 50\%$  (중간):

└─ Dynamic Merge 추천

1. 벡터 검색과 메타데이터 필터 병렬 진행
2. 조건에 맞는 결과가 나올 때까지 계속

$\sigma > 90\%$  (매우 높음):

└─ Search-then-Filter 추천

1. 모든 항목에서 벡터 검색
2. 검색 결과 중에서 필터 적용

### 전략 선택의 비용:

Filter-then-Search 비용:

= 필터 스캔 비용 +  $(1-\sigma)N \times$  벡터검색 비용

Search-then-Filter 비용:

= 모든항목 벡터검색 비용 + 필터 검사 비용

Dynamic Merge 비용:

= min(위 두 방식 비용)

## 76.4 선택도 추정과의 연관성

### 정확한 선택도 추정의 중요성:

실제 선택도: 15%

추정된 선택도: 50%

결과:

- Dynamic Merge 선택
- 실제로는 Filter-then-Search 가 2 배 빨랐음
- 성능 저하 발생

### Exqutor 와의 연계:

Exqutor 의 선택도 추정이 정확해야 ACORN 의 동적 병합 전략이 올바른 결정을 내릴 수 있다.

## 76.5 복합 술어 처리

### 기본 술어:

```
price < 500      (범위)
category = 'phone' (등호)
rating > 4.5     (범위)
```

### 복합 술어:

```
(price < 500 AND category = 'phone') OR rating > 4.9
```

이를 정규 형식으로:

DNF (Disjunctive Normal Form):

```
(price < 500 AND category = 'phone')
OR (rating > 4.9)
```

### 선택도 계산:

```
 $\sigma(A \text{ OR } B) = \sigma(A) + \sigma(B) - \sigma(A \text{ AND } B)$ 
= 15% + 2% - 0.3%
= 16.7%
```

## 76.6 실험 결과

### 벤치마크 설정:

- 데이터셋: Amazon products (100M), NYC taxi (100M)
- 쿼리: 다양한 선택도의 필터 조건
- 비교: Filter-then-Search, Search-then-Filter, ACORN

### 결과 (쿼리 응답 시간):

| 선택도 | FTS   | STF   | ACORN | 최적          |
|-----|-------|-------|-------|-------------|
| 5%  | 50ms  | 800ms | 52ms  | FTS (50ms)  |
| 25% | 200ms | 300ms | 210ms | STF (300ms) |
| 50% | 350ms | 350ms | 360ms | 동등          |
| 75% | 500ms | 200ms | 210ms | STF (200ms) |
| 95% | 600ms | 100ms | 102ms | STF (100ms) |

### ACORN의 성능:

- 최적 방식의 95-105% 성능 유지
- 선택도 미리 알 수 없는 상황에서 일반적 우수 성능

## 76.7 본 논문과의 관계

### Exqutor의 전략 선택 문제:

Exqutor도 유사한 딜레마에 직면한다:

범위 필터와 벡터 검색의 결합

선택도  $\sigma$ 가 높음 (대부분 벡터가 필터 통과):

└─ 전체 벡터 인덱스 사용해 검색 후 필터링

선택도  $\sigma$ 가 낮음 (매우 적은 벡터만 필터 통과):

└─ 먼저 필터 실행 후 남은 벡터만 인덱싱

### 정확한 선택도 추정의 중요성:

- 잘못된 전략 선택 시 성능 저하
- 선택도 추정 오류  $\pm 20\%$  범위 내 유지 필요

### 동적 최적화:

- 쿼리 특성에 따라 전략 동적 변경
- 적응적 알고리즘으로 최악의 경우 회피

## 추가 제기 문제

1. **선택도 오류의 민감도:** 선택도 추정이  $\pm 50\%$  오류일 때, ACORN의 성능 저하는 얼마나 되는가?
2. **복잡한 복합 술어:** AND, OR, NOT이 중첩된 매우 복잡한 술어에서 선택도를 정확히 추정할 수 있는가?
3. **시간 경향(Temporal Trend):** 시간대별로 데이터 분포가 변할 때, 선택도 추정을 어떻게 적응시킬 것인가?
4. **다중 인덱스 활용:** 여러 차원에 인덱스가 있을 때, 어느 인덱스를 우선 사용할지 결정하는 방법은?
5. **병렬 처리:** 벡터 검색과 메타데이터 필터링을 병렬로 실행할 때의 오버헤드는?
6. **점진적 결과 반환:** 모든 결과를 찾기 전에 부분 결과를 사용자에게 스트리밍할 수 있는가?